



DATA MINING & PREDICTIVE ANALYTICS | PROMPT LOGBOOK

Mandi @ 36: Working with AI

Prompt Logbook for Customer Experience Intelligence

Submitted by	Malla Venkata Sai Ashish
Roll Number	B31-25
Project	Mandi @ 36, Hyderabad

Evidence-grounded analytics, controlled AI assistance, and human validation

Purpose and Disclosure

This logbook records the prompts used or reconstructed from the actual Mandi @ 36 Data Mining and Predictive Analytics workflow. AI tools supported code scaffolding, method explanations, dashboard communication, quality assurance, and controlled analyst responses. They did not create raw Google Maps reviews, replace deterministic analytics, or manufacture numerical findings.

Evidence rule. Before any AI-assisted narrative or code was adopted, it was checked against the local data schema, pipeline artifacts, dashboard outputs, and project code. Raw Apify exports remain separate and read-only.

Prompt Records

The record combines the prompt, the intended tool, a concise output summary, and the validation or project use. Prompts are written so a reviewer can reproduce the reasoning path without exposing unnecessary customer-review data to a language model.

No.	Tool and task	Prompt text	Output, project use, and reflection
1	Codex Schema inspection	I have Apify Google Maps scraper JSON exports for three Mandi @ 36 branches. Inspect the actual schema without modifying any raw file. Identify the review-list field and map branch, review text, rating, date, reviewer, and source URL. Report missing or ambiguous fields before proposing code.	Output: field-mapping checklist and validation plan. Use: confirmed the data contract before coding. Reflection: schema inspection prevented assumptions about nested records and preserved raw files.
2	GitHub Copilot JSON preparation	Convert the inspected Google Maps scraper JSON into a usable analytical table in Python. Use a defensive loader for nested review arrays; retain branch and source URL; normalize rating and date; preserve raw text; and write a separate cleaned output. Do not invent missing values.	Output: loader scaffold and normalization logic. Use: adapted after checking the actual exports. Reflection: the cleaned dataset was written separately, so the transformation remained reproducible and reversible.

No.	Tool and task	Prompt text	Output, project use, and reflection
3	Claude Text preparation	Suggest a privacy-minimizing cleaning sequence for public restaurant reviews. Preserve the original review text, create a clean analysis field, and list limits for emojis, mixed languages, sarcasm, empty text, and informal spelling.	Output: text-cleaning steps and limitations. Use: informed the preparation section and the rule to treat textual outputs as review cues, not definitive customer truth.
4	Codex Sentiment baseline	Create a transparent sentiment baseline for restaurant review text using interpretable positive and negative lexicons. Return a label and score, keep it deterministic and separate from LLM calls, and state why it must not be presented as CSAT or NPS.	Output: deterministic baseline design. Use: supported derived sentiment labels and dashboard wording. Reflection: the baseline is useful for triage but remains unvalidated and requires review of original comments.
5	ChatGPT Topics and aspects	Explain, in plain language, how keyword aspects plus TF-IDF and NMF topic modeling can identify recurring restaurant experience areas such as food, service, ambience, waiting time, and value. Include limitations and manager-validation steps.	Output: readable methodology explanation. Use: supported report and dashboard explanations. Reflection: a topic or aspect frequency directs investigation; it does not prove a root cause.
6	GitHub Copilot Data mining	Propose reproducible Python steps for review clustering and association-rule analysis after data cleaning. Keep output artefacts separate, provide parameters in code comments, and describe how a manager should interpret groups and co-occurrences without treating them as causal findings.	Output: code and interpretation scaffold. Use: informed cluster, recurring review-group, and association views. Reflection: groups summarize patterns in the available sample and must be checked against real reviews.
7	Codex Predictive monitoring	Design a transparent monthly monitoring workflow for branch ratings. Produce observed trend, linear forecast, uncertainty display, and anomaly flags. Clearly label forecasts as unvalidated and recommend manager review rather than automatic intervention.	Output: monitoring design and caveats. Use: informed Predictive Analytics and Anomaly Center views. Reflection: early warnings identify where to look, not what definitely happened or will happen.
8	Claude Dashboard design	For a Streamlit restaurant-review intelligence app, propose a logical set of pages that helps a manager explore executive overview, sentiment, themes, mining, forecasts, anomalies, branch intelligence, and AI-assisted questions. Keep collected, derived, and predicted evidence visibly distinct.	Output: information architecture and visual roles. Use: shaped the page structure and labels of the deployed dashboard. Reflection: the pages reduce cognitive load by separating descriptive, mined, and predictive evidence.

No.	Tool and task	Prompt text	Output, project use, and reflection
9	Project AI Analyst Scenario 1	How is Banjara Hills performing and what actions should we take? Use only compact JSON evidence for branch summary, trends, recurring aspects, topics, anomaly signals, and recommended actions. State the evidence basis, uncertainty, and what a manager should validate.	Output: grounded branch investigation response. Use: demonstrated the AI Analyst page and report scenario. Reflection: the orchestrator routes the query and supplies compact evidence; the manager remains accountable for action.
10	Project AI Analyst Scenario 2	Compare Banjara Hills and Gachibowli. Which branch needs priority attention and why? Use only supplied branch summaries, review volume, rating trends, anomaly status, and recurring themes. Do not claim causality or make unsupported ranking statements.	Output: comparative evidence narrative with cautions. Use: demonstrated cross-branch prioritisation. Reflection: unequal review volumes and observational data require context before a management decision.
11	ChatGPT Business implications	Translate evidence from sentiment, topics, clusters, forecasts, and anomalies into cautious restaurant-management implications. Separate observed facts, derived signals, predicted indicators, and recommended actions. Do not add metrics that are not present in the supplied evidence.	Output: recommendation structure. Use: supported business-implication writing. Reflection: recommendations were grounded in project artefacts and clearly distinguished from proven outcomes.
12	Codex Submission QA	Check the DMPA submission against this brief: business objectives, methodology, AI details, step-by-step prompts and outputs, results and discussion, business implications, visuals, runnable links, references, and no unsupported claims. Identify evidence-backed corrections only.	Output: final QA checklist. Use: supported report, slides, and logbook review. Reflection: visual rendering and local-artifact checks were used before finalizing the submission files.

Verification and Reflection

The project uses deterministic local analytics for all numerical evidence, including dataset and branch summaries, NLP outputs, association rules, clusters, anomaly results, and forecast outputs. AI assistance helped make the workflow more usable and explainable, but it was never treated as a substitute for data validation. The strongest practice is to preserve the original evidence, label derived and predicted outputs, send only compact evidence to the AI Analyst, and require a human manager to validate recommendations.

Technical Evidence Used

Data pipeline. src/pipeline.py reads raw exports, normalizes reviews, runs deterministic analytics, and writes output artefacts without modifying raw files.

Data mining. The project produces transparent topic, aspect, cluster, association, anomaly, and forecast artefacts under results/tables for review and reuse.

AI Analyst. src/agents/orchestrator.py detects branch, comparison, period, and action intent before passing compact evidence to the response layer.

Verification. The live dashboard, project reports, slides, repository, and Google Maps branch sources were cross-checked before submission.

Access and Prompt-Control Checklist

The following controls make future prompt use repeatable and suitable for an academic analytics project. They are also the practical boundary between AI assistance and the deterministic evidence base.

Control	Required practice
Task scope	Use one analytical question per prompt and state the required output clearly.
Data boundary	Inspect schema locally; share compact, de-identified evidence rather than raw review exports.
Evidence label	Separate collected facts, derived analytics, predictions, and recommendations in every answer.
Validation	Check generated code and narrative against pipeline artefacts, dashboard views, and source review samples.
Decision rights	Keep branch actions, customer communication, and operational decisions under accountable human review.

Repository: <https://github.com/im45145v/Mandi360>

Live dashboard: <https://mandi36.streamlit.app/>